Jose Pablo Canton Monzón
26009
Programación Orientada a Objetos (POO)

Análisis del Ejercicio 4

Requisitos Funcionales y Reglas de Negocio

Información a conservar:

- **Común a todo vehículo:** Placa (identificador único), marca, modelo, tarifa diaria y estado de disponibilidad (alquilado o disponible).
- **Automóviles:** Cantidad de pasajeros y si la transmisión es automática o manual.
- **Motocicletas:** Cilindraje en centímetros cúbicos (cc).
- **Camionetas de carga:** Capacidad máxima de transporte en toneladas (incluye decimales).
- **Empresa:** Dinero acumulado por alquileres confirmados.

Operaciones del sistema:

- **Registro:** Incorporar vehículos nuevos al catálogo. Cada vehículo ingresa automáticamente con estado "disponible".
- **Cotización:** Localizar un vehículo por su placa y calcular el costo por una cantidad específica de días. Esta acción muestra las características, disponibilidad y costo, pero no modifica los ingresos ni el estado del vehículo.
- **Alquiler:** Confirmar la transacción si el vehículo está disponible. Cambia el estado a ocupado y suma el cobro al ingreso total acumulado. No se permiten dos alquileres simultáneos sobre el mismo vehículo.
- **Devolución:** Buscar por placa y cambiar el estado del vehículo a disponible. No genera nuevos cobros ni resta el ingreso previo.
- **Reportes:** Consultar cantidad de vehículos registrados, disponibles y alquilados por cada categoría, además del dinero total recaudado.

Condiciones y Validaciones:

- No pueden existir dos registros con la misma placa. No se aceptan placas vacías.
- Las tarifas, cantidades de pasajeros, cilindrajes y capacidades deben ser mayores que cero.
- Los días de alquiler deben ser números enteros positivos.
- No se puede alquilar un vehículo ocupado ni devolver uno que ya está disponible.
- Entradas con formato incorrecto no deben terminar la ejecución del programa de forma inesperada.

Propuesta de Clases y Responsabilidades

1. Clase Vehiculo (Clase Abstracta / Superclase)

- **Responsabilidad:** Contener los datos y comportamientos comunes a toda la flota, definiendo un contrato para el cálculo del costo de alquiler.
- **Atributos:**
    - placa (String, *protected/private*): Identificador único.
    - marca, modelo (String, *protected/private*): Datos de identificación.
    - tarifaDiaria (double, *protected/private*): Base de cobro.
    - disponible (boolean, *protected/private*): Indica si el vehículo puede ser alquilado.
- **Métodos principales:**
    - **Constructor:** Recibe placa, marca, modelo y tarifa. Inicializa el atributo disponible internamente en true.
    - calcularCostoTotal(dias: int) -> double: **Método abstracto** que obliga a las subclases a implementar su propia lógica de cobro.
    - Getters/Setters estándar para acceder a la disponibilidad y a la placa.

2. Clase Automovil (Subclase de Vehiculo)

- **Responsabilidad:** Modelar los automóviles familiares y aplicar su recargo por transmisión.
- **Atributos:** capacidadPasajeros (int, *private*), esAutomatica (boolean, *private*).
- **Métodos principales:**
    - **Constructor:** Recibe todos los atributos comunes más los específicos e invoca al constructor de la superclase (super).
    - calcularCostoTotal(dias: int): Si esAutomatica es true, suma Q50 a la tarifaDiaria antes de multiplicar por los días; si es manual, solo multiplica la tarifa base por los días.

3. Clase Motocicleta (Subclase de Vehiculo)

- **Responsabilidad:** Modelar motocicletas y aplicar recargos basados en la cilindrada.
- **Atributos:** cilindraje (int, *private*).
- **Métodos principales:**
    - **Constructor:** Invoca al constructor de la superclase y asigna el cilindraje.
    - calcularCostoTotal(dias: int): Multiplica la tarifa base por los días. Si cilindraje es mayor a 250, suma un cargo único y fijo de Q75 al total resultante.

4. Clase CamionetaCarga (Subclase de Vehiculo)

- **Responsabilidad:** Modelar vehículos de carga y aplicar el recargo escalable por tonelaje.
- **Atributos:** capacidadToneladas (double, *private*).
- **Métodos principales:**
    - **Constructor:** Invoca a super y asigna la capacidad.
    - calcularCostoTotal(dias: int): Por cada día, suma a la tarifa diaria base el resultado de (capacidadToneladas * Q100). Luego multiplica todo por la cantidad de días.

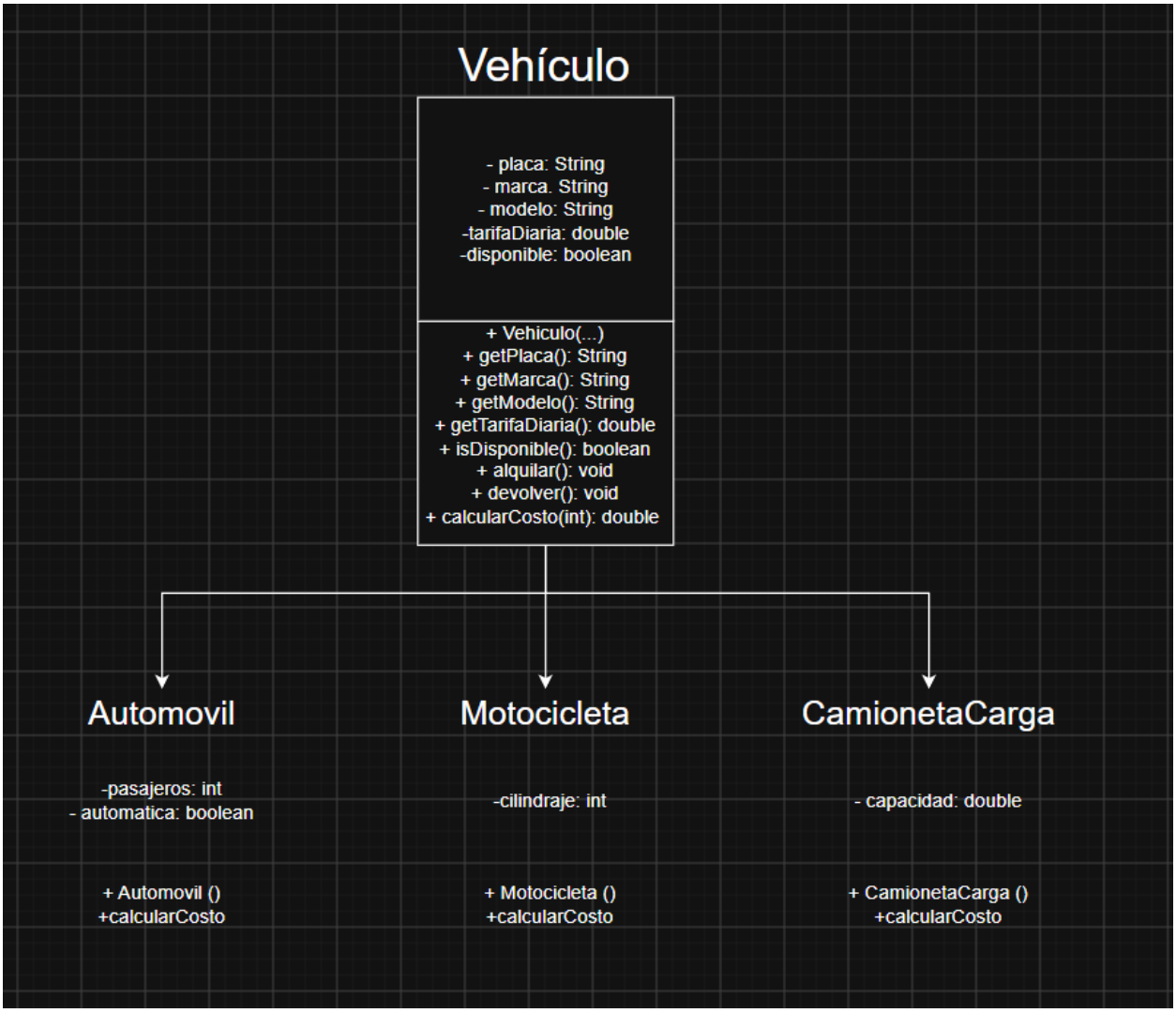
Justificación del Diseño

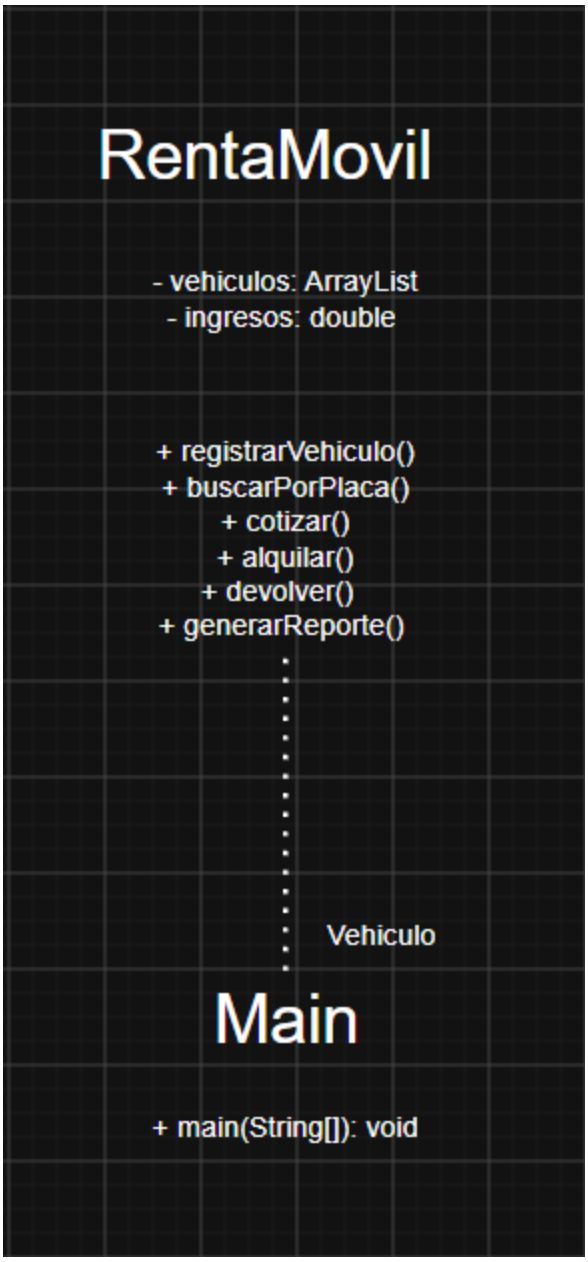
Organización de elementos y colaboración: La información compartida (placa, tarifa, estado) reside en la superclase Vehiculo, eliminando la duplicación de código y estandarizando los atributos esenciales de la flota. Las variaciones (pasajeros, toneladas, transmisiones) se encapsulan únicamente en las subclases pertinentes.

Las decisiones de cobro ocurren **dentro de cada subclase** (en el método calcularCostoTotal), lo que delega la responsabilidad a cada categoría en lugar de tener estructuras condicionales masivas en la clase gestora. La clase controladora (EmpresaRenta) simplemente colabora con los objetos enviando el mensaje polimórfico "calcula tu costo", desconociendo internamente las fórmulas matemáticas que aplica cada tipo de vehículo.

Escalabilidad ante nuevas categorías: Si en el futuro la empresa desea agregar, por ejemplo, minivans o bicicletas eléctricas con diferentes reglas de cobro, el sistema actual está preparado para extenderse de forma limpia. Únicamente habría que crear una nueva clase que herede de Vehiculo y sobreescribir el método calcularCostoTotal(int dias) para reflejar la nueva regla. No sería necesario modificar los métodos de cotización, alquiler o devolución de la clase controladora, garantizando el cumplimiento del principio Abierto/Cerrado (Open/Closed Principle) de la programación orientada a objetos.

UML


Vehículo
- placa: String
- marca. String
- modelo: String
-tarifaDiaria: double
-disponible: boolean
+ Vehiculo(...)
+ getPlaca(): String
+ getMarca(): String
+ getModelo(): String
+ getTarifaDiaria(): double
+ isDisponible(): boolean
+ alquilar(): void
+ devolver(): void
+ calcularCosto(int): double
Automovil
-pasajeros: int
- automatica: boolean
+ Automovil ()
+calcularCosto
Motocicleta
-cilindraje: int
+ Motocicleta ()
+calcularCosto
CamionetaCarga
- capacidad: double
+ CamionetaCarga ()
+calcularCosto


RentaMovil
- vehiculos: ArrayList
- ingresos: double
+ registrarVehiculo()
+ buscarPorPlaca()
+ cotizar()
+ alquilar()
+ devolver()
+ generarReporte()
Vehiculo
Main
+ main(String[]): void

Pruebas de funcionamiento

```
================================
          RENTAMOVIL
================================
1. Registrar vehículo
2. Consultar flota
3. Cotizar alquiler
4. Confirmar alquiler
5. Registrar devolución
6. Mostrar reporte general
0. Salir
================================
Seleccione una opción: 1

===== REGISTRAR VEHÍCULO =====
1. Automóvil
2. Motocicleta
3. Camioneta de carga
Seleccione el tipo: 1
Placa: P5593B
Marca: KIA
Modelo: SEDEN
Tarifa diaria: 155.00
Cantidad de pasajeros: 5
¿La transmisión es automática?
1. Sí
2. No
Seleccione: 1
Vehículo registrado correctamente.
```